What I Brought Back From CodeStock

A Lunch & Learn

Bring your lunch, bring an opinion.

What is CodeStock?

- A **community-run** tech conference in **Knoxville, TN**
- Sessions across languages, process, AI and soft skills
- I attended four sessions, and each became a Takeaway:

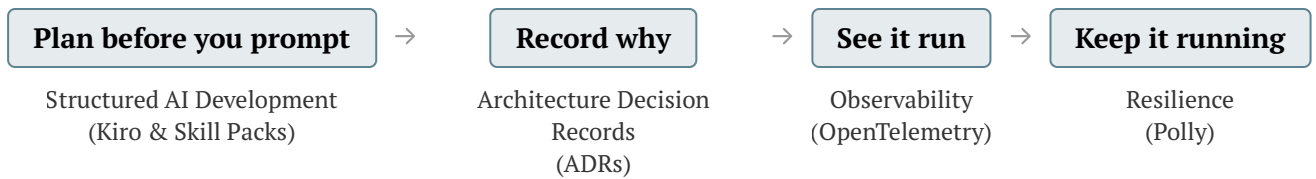
What the Spec! Battle-Tested Practices for Spec-Driven Development Using Kiro
Chris Gonzalez

Navigating the Maze of Communicating Architecture Decisions
Chad Green

Shift Observability Left: Why It Starts in Dev
Victor Lyuboslavsky

Polly Want a 500? Chaos Engineering for .NET Apps
Victor Frye

Agenda: The Arc



then: **Discussion**

Structured AI Development

Kiro & Skill Packs

Plan before you prompt → Record why → See it run → Keep it running

From *What the Spec! Battle-Tested Practices for Spec-Driven Development Using Kiro*, Chris Gonzalez

What it is: specs, not vibes

Give AI-assisted coding a deliberate workflow, so you plan before you prompt.



Each step is a Markdown file a human reviews and edits *before* any code is written.

The problem it solves

- Ad-hoc prompting produces code **nobody planned**, so nobody can really **review** it
- The agent fills every gap with a guess, and the guesses only show up later
- Context lives in one chat window, then it’s gone

"The work you do up front matters 10x more."

Chris Gonzalez

Same building blocks, packaged differently

Building block	Kiro: kiro-quickstart	General-purpose agent: this repo
Steering: standing guidance	`.kiro/steering/`	`AGENTS.md`
Skills: reusable workflows	`.kiro/skills/`	Skill Packs: Matt Pocock's skills, pstack
Hooks: automation on events	`.kiro/hooks/` (quality checks, docs sync)	*empty: the next thing to try*
Specs: requirements → design → tasks	`.kiro/specs/` (on the `demo/*` branches)	`.kiro/specs/codestock-lunch-and-learn-deck/`

Kiro column: Chris Gonzalez's example repo, github.com/chrismgonzalez/kiro-quickstart

Dogfood: this deck

A grilling session → `CONTEXT.md`

The agent interviewed me (Matt Pocock's skills) and wrote down the vocabulary we settled on:

```
**Discussion**:
The open part of the Lunch & Learn after the
Highlights, where attendees bring their own takeaways,
questions and opinions.
_Avoid_: Q&A

**Discussion Prompt**:
The deliberately provocative question that ends each
Takeaway. All four are collected again to open the
Discussion, and the room picks which one to start with.
_Avoid_: question, talking point
```

A Kiro-*style* spec

The same requirements → design → tasks flow, written by a general-purpose agent. No Kiro involved.

```
.kiro/specs/codestock-lunch-and-learn-deck/
├── requirements.md   user stories + EARS criteria
├── design.md         files, components, edge cases
└── tasks.md          ordered, checkable tasks
```

A real requirement, in EARS

Easy **A**pproach to **R**equirements **S**yntax: *WHEN* ‹trigger› *THE* ‹system› *SHALL* ‹response›

```
- [ ] 2.3 WHEN the Takeaway shows the spec THE DECK SHALL include at least one real EARS acceptance criterion from this file.
```

This slide is the one that satisfies it.

Imported from `requirements.md` at build time

The Six Principles

From Chris Gonzalez's session

1. Understand the behavior.
2. **You're now an architect and decision maker.**
3. The work you do up front matters 10x more.
4. Review, review, review.
5. You're still responsible for AI-written code.
6. Discipline = freedom.

First Step

Install **one** Skill Pack in the agent your project already uses, and run it on your next ticket.

Worked example, Codex: put your project's conventions in `AGENTS.md` (steering), and install the skills wherever your agent loads them.

Discussion Prompt

With AI writing more of the code, “you’re now an architect and decision maker.” Agree or disagree?

That grilling session also produced this repo’s ADRs →

Which is where we're going next.

Architecture Decision Records

ADRs

Plan before you prompt → **Record why** → See it run → Keep it running

From *Navigating the Maze of Communicating Architecture Decisions*, Chad Green

What it is

A short text file that records **one** significant decision, and **why** it was made.

Title	**Status**	**Context**	**Decision**	**Consequences**
a short noun phrase	proposed, accepted, superseded	the forces at play	what we will do	good, bad and neutral

- Lives in the repo, next to the code, numbered in order: `docs/adr/0001-….md`
- Never edited after the fact: a new ADR **supersedes** an old one
- **A one-paragraph ADR is valid.** Extra sections, such as Considered Options, are optional.

The problem it solves

Decisions outlive the people who remember *why* they were made.

When a newcomer doesn't know why, they have two bad choices:

- **Blindly accept it**, even after the reasons stop being true
- **Blindly change it**, and break the thing it was protecting

After Michael Nygard, *Documenting Architecture Decisions* (2011)

A strong ADR: this deck’s ADR 0001

```
---
status: accepted
date: 2026-10-01
deciders: Dave Staab
---

# Build the deck in Slidev, not PowerPoint or Google Slides

## Context

The Lunch & Learn shares four Takeaways from CodeStock. Most of its
examples are code: OpenTelemetry setup, Polly resilience pipelines,
and spec and ADR files. Two of the Takeaways (Structured AI
Development and ADRs) are meant to be Dogfooded: the deck itself
should be built with the practices it presents, so the examples are
real.

## Decision

Build the deck in [Slidev](https://sli.dev), as Markdown slides in a
git repo, alongside its own `CONTEXT.md`, Kiro-style spec and ADRs.

## Considered Options

- **PowerPoint**: familiar to everyone, and easy for people who don't
code to edit and comment on. Code has to be pasted in as images or
poorly formatted text, and there's nowhere natural for specs and ADRs
to live.
- **Google Slides**: the same trade-offs as PowerPoint, plus easy
sharing. It has the same weakness with code.
- **Slidev** (chosen): code highlighting built in (Shiki), slides are
plain text that an AI agent can read and edit, and the deck's spec,
glossary and ADRs live in the same repo as the slides.

## Consequences

- Good: code snippets look right without extra effort, and an AI
agent can work on the deck the same way it works on code.
- Good: the Dogfood is honest. The ADRs and spec shown on the slides
are the real files that shaped the deck.
- Bad: managers, QA and designers can't easily edit or comment on the
slides. Feedback has to come through the presenter or a PDF export.
- Bad: moving to another tool later means rebuilding every slide by
hand.
- Follow-on: sharing the deck needs a hosting decision (see [ADR
0002](0002-netlify-and-pdf-for-sharing.md)).
```

A borderline ADR: 0002

```
## Decision

Host the deck on Netlify, and delete `vercel.json`.
Netlify is the default because the presenter already
uses it for other projects; another host would need a
compelling feature to win. Also export a PDF (`pnpm run
export`) for people who want a file they can keep or
attach.
```

Write an ADR only if it is all three:

1. **Hard to reverse**: ✕ **0002 fails: switching hosts is one config file**
2. **Surprising without context**: ✓ why not Vercel?
3. **A real trade-off**: ✓ an existing account vs. a host with no advantage

0002 says so itself, in its Consequences:

```
- Neutral: switching to Vercel later is cheap (one
config file), which is why this ADR is short on drama.
It is recorded mainly as a teaching example.
```

Smaller choices belong in the spec.

First Step

Write an ADR for the **next** real decision your team makes.

One paragraph is enough: what you decided, and why. Commit it to `docs/adr/0001-….md`.

Discussion Prompt

Name a decision in our codebase nobody can explain anymore.

What would it have cost to write it down?

Observability

OpenTelemetry

Plan before you prompt → Record why → **See it run** → Keep it running

From *Shift Observability Left: Why It Starts in Dev*, Victor Lyuboslavsky

Shift Left

Observability starts in **dev**, not in production.

Add telemetry from day one, so dev and prod tell the same story.

What it is

Making a system's internal behavior visible, using the **OpenTelemetry** standard.

Metrics

Measurements over time, such as request counts and timings. The summary view.

Logs

Timestamped records with the details.

Traces

Follow one request through the system, so it leaves a trail.

The trace gives the map; the logs give the notes.

The problem it solves

Code intent ≠ runtime behavior

"The response was correct, but the path it took was not."

Without telemetry, you only ever see the response.

The pipeline



RECURRING SIGNAL

The **Aspire dashboard** came up in *both* the observability and the resilience sessions, independently, as the place to start before production tooling.

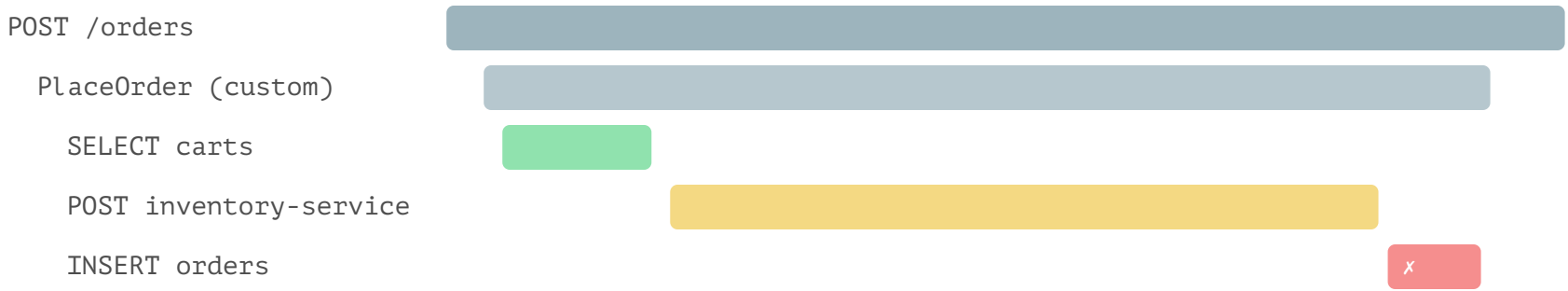
Turn it on: tracing setup

```
// Program.cs (OrdersApi)
// Packages: OpenTelemetry.Extensions.Hosting, OpenTelemetry.Instrumentation.AspNetCore,
//           OpenTelemetry.Instrumentation.Http, OpenTelemetry.Exporter.OpenTelemetryProtocol
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddOpenTelemetry()
    .ConfigureResource(resource => resource.AddService("OrdersApi"))
    .WithTracing(tracing => tracing
        .AddAspNetCoreInstrumentation()   // a span for every incoming request
        .AddHttpClientInstrumentation()   // a child span for every outgoing call
        .AddSource("OrdersApi")           // our own custom spans
        .AddOtlpExporter());              // send to localhost:4317 by default

var app = builder.Build();
```

Anatomy of a trace



1. A request starts a span. Most frameworks already do this.
2. Child spans come from libraries, or you add your own.
3. DB calls and outgoing HTTP calls get their own spans. Errors are marked on the span.

Latency has an address: the trace shows where the time goes.

Your own spans

```
using System.Diagnostics;

public class CheckoutService(InventoryClient inventory)
{
    private static readonly ActivitySource Source = new("OrdersApi");

    public async Task<Order> PlaceOrderAsync(Cart cart)
    {
        using var activity = Source.StartActivity("PlaceOrder");
        activity?.SetTag("order.item_count", cart.Items.Count);

        try
        {
            await inventory.ReserveAsync(cart.Items); // child span via HttpClient
            return Order.From(cart);
        }
        catch (Exception ex)
        {
            activity?.SetStatus(ActivityStatusCode.Error, ex.Message);
            throw;
        }
    }
}
```

First Step

Run the Aspire dashboard locally and look at **one** trace.

```
# UI on http://localhost:18888; OTLP on 4317 (gRPC) and 4318 (HTTP).
# The login token is printed in the container logs: docker logs aspire-dashboard
docker run --rm -d -p 18888:18888 -p 4317:18889 -p 4318:18890 \
  --name aspire-dashboard mcr.microsoft.com/dotnet/aspire-dashboard:latest
```

Point your app's OTLP exporter at it (the default `localhost:4317` already does), make one request, and open `http://localhost:18888`.

Discussion Prompt

Do our dev and prod environments tell the same story?

Resilience

Polly

Plan before you prompt → Record why → See it run → **Keep it running**

From *Polly Want a 500? Chaos Engineering for .NET Apps*, Victor Frye

> Victor Frye's session was about chaos engineering. We're focusing on resilience, and using chaos only to prove it works.

What it is

Keeping your system working when the things it depends on fail.

Retry

It failed? Wait a moment and try again. Many failures are temporary.

Circuit breaker

It keeps failing? Stop calling it for a while, so you don't make things worse.

Timeout

It's taking too long? Give up, rather than leave the user waiting.

Fallback

Still failing? Return something sensible instead, such as cached data.

Polly is the .NET library for all of these.

The problem it solves

Every remote call can fail.

An unprotected one becomes **your** outage.

The network, a database, a partner API, your own other service.

One line of resilience

```
// Program.cs (OrdersApi)
// Package: Microsoft.Extensions.Http.Resilience (built on Polly)
builder.Services
    .AddHttpClient<InventoryClient>(client =>
        client.BaseAddress = new("https://inventory-service"))
    .AddStandardResilienceHandler();

// One line gives you, outermost to innermost:
//   rate limiter → total timeout (30s) → retry (3x, exponential + jitter)
//   → circuit breaker → attempt timeout (10s)
```

Prove it works: inject a fault

```
using System.Net;
using Polly.Simmy;

var inventory = builder.Services
    .AddHttpClient<InventoryClient>(client =>
        client.BaseAddress = new("https://inventory-service"));

inventory.AddStandardResilienceHandler();   // 1. resilience first (outer)

if (builder.Environment.IsDevelopment())
{
    // 2. chaos after it (inner): 5% of calls come back as a 500
    inventory.AddResilienceHandler("chaos", pipeline => pipeline
        .AddChaosOutcome(0.05, () =>
            new HttpResponseMessage(HttpStatusCode.InternalServerError)));
}
```

The caller almost never sees the 500s: the standard handler retries them. Polly's telemetry shows each one, so watch it happen in the Aspire dashboard.

First Step

Add the standard resilience handler to one flaky `HttpClient`.

One package, one line. Then add the chaos handler in dev, and watch it hold up.

Discussion Prompt

Which dependency, if it went down for 10 minutes, would take us down with it?

Discussion

Which one do we start with? Show of hands.

1. STRUCTURED AI DEVELOPMENT

   With AI writing more of the code, “you’re now an architect and decision maker.” Agree or disagree?

2. ARCHITECTURE DECISION RECORDS

   Name a decision in our codebase nobody can explain anymore. What would it have cost to write it down?

3. OBSERVABILITY

   Do our dev and prod environments tell the same story?

4. RESILIENCE

   Which dependency, if it went down for 10 minutes, would take us down with it?

Resources

Structured AI Development

- Kiro: kiro.dev
- Chris Gonzalez's example repo: github.com/chrismgonzalez/kiro-quickstart
- Matt Pocock's skills: github.com/mattpocock/skills
- pstack, by Lauren Tan (poteto): link to confirm

ADRs

- Michael Nygard, *Documenting Architecture Decisions*: cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Observability

- OpenTelemetry .NET: opentelemetry.io/docs/languages/dotnet
- Aspire dashboard (standalone): aspire.dev/dashboard/standalone

Resilience

- Polly: pollydocs.org
- HTTP resilience in .NET: learn.microsoft.com/dotnet/core/resilience/http-resilience

This deck, its `CONTEXT.md`, spec and ADRs are all in its repo.